

Accelerating Option Pricing with Neural Network Surrogates: From Black-Scholes to Stochastic Volatility

Bhavya Shah

Independent Quantitative Finance and Machine Learning Research Project

August 31, 2026

Abstract

This paper studies neural networks as computational surrogates for derivative pricing. The central question is not whether a neural network can reproduce an option-pricing function, but when the upfront cost of learning that function is justified by repeated use. The analysis begins with Black-Scholes as a controlled benchmark and compares analytical valuation, Monte Carlo simulation, and a feed-forward neural network trained on synthetic option-pricing data. The framework is then extended to the Heston stochastic-volatility model, where the reference pricing operator is numerical and therefore provides a stronger motivation for surrogate modeling. The final Heston experiment uses 8,000 synthetic parameter vectors and Monte Carlo labels generated with 5,000 antithetic paths and 100 time steps. The surrogate is a 9-128-128-64-32-1 multilayer perceptron trained with Huber loss for up to 1,000 epochs with early stopping. Across three random seeds, mean test-set MAE is 0.3132 (95% CI: 0.2748-0.3516) and mean RMSE is 0.4093 (95% CI: 0.3613-0.4574); the selected model has MAE 0.2968 and RMSE 0.3883. On a 100-option high-accuracy revaluation using 20,000 antithetic paths and 200 time steps, the selected surrogate attains MAE 0.2745 and RMSE 0.3561. In a repeated-pricing benchmark using compiled antithetic Monte Carlo, inference is approximately 1,265 times faster, with a bootstrap 95% interval of 1,162-1,358 times. Accounting for 4.58 seconds of one-time training, the estimated amortized break-even point is approximately 8,084 pricing queries. The evidence supports a conditional conclusion: neural-network surrogates are not compelling replacements for closed-form Black-Scholes pricing, but can materially reduce repeated valuation cost for numerically priced stochastic-volatility models when approximation error is controlled and the surrogate is reused sufficiently often.

Keywords: option pricing; neural networks; surrogate models; Black-Scholes; Heston; Monte Carlo; computational finance

JEL classification: C14, C45, C53, G13, G17

1. Introduction

Derivative pricing provides a useful setting in which to study the interaction between quantitative finance and machine learning. The Black-Scholes framework offers a closed-form benchmark under restrictive assumptions, while stochastic-volatility models such as Heston typically require numerical valuation. This difference creates a natural test of when a learned approximation is economically useful: the relevant comparison is not simply prediction accuracy or inference speed, but the total cost of training and repeatedly evaluating the pricing operator.

The idea of learning derivative-pricing relationships with neural networks is not new. Hutchinson, Lo, and Poggio [1] demonstrated that learning networks could recover option-pricing relationships from simulated data and emphasized that network pricing is most compelling when the underlying pricing equation is unavailable or difficult to solve analytically. More recent work has developed increasingly sophisticated approaches to the same computational problem. Hainaut and Casas [5] use physics-inspired neural networks for Heston pricing, while Funahashi [6] studies residual-learning and quasi-process corrections for derivative pricing. Chen, Didisheim, and Scheidegger [7] develop deep surrogate models for high-dimensional financial structural models and show how learned surrogates can reduce repeated model-evaluation costs. The present study deliberately takes a narrower and more transparent approach. Rather than introducing a large architecture or an arbitrage-constrained neural PDE solver, it asks whether a modest feed-forward network can act as a reusable numerical-pricing surrogate and whether the computational advantage survives after training cost and timing uncertainty are included. Black-Scholes serves as a controlled negative benchmark: replacing an inexpensive closed-form expression with a neural network should not be expected to

create a meaningful computational benefit. Heston provides the main use case because the reference price is generated through numerical simulation.

Three research questions guide the analysis.

1.1. Research Questions

- RQ1. Can a feed-forward neural network accurately learn the Black-Scholes pricing surface from synthetic examples?
- RQ2. Does the computational case for a neural-network surrogate become stronger when pricing requires numerical simulation under stochastic volatility?
- RQ3. After accounting for training cost and uncertainty in the timing benchmark, how many repeated pricing queries are required before the surrogate becomes computationally attractive?

1.2. Contributions

- A transparent comparison of analytical, Monte Carlo, and learned option valuation.
- An explicit amortized-cost framework that treats training as an economic cost rather than reporting inference speed in isolation.
- A Heston experiment with higher-accuracy training labels, multi-seed robustness, error decomposition by moneyness and maturity, and an independent high-accuracy Monte Carlo revaluation.
- A reproducible benchmark using compiled antithetic Monte Carlo with repeated timing and bootstrap uncertainty for the reported speed-up.

The remainder of the paper is organized as follows. Section 2 reviews the pricing and surrogate-model literature. Section 3 presents the theoretical pricing framework. Section 4 describes the synthetic data, learning architecture, evaluation metrics, and computational benchmark. Section 5 reports the empirical

results. Section 6 discusses their interpretation. Section 7 considers robustness and limitations, and Section 8 concludes.

2. Literature Review

2.1. Classical Option Pricing and Numerical Valuation

Black and Scholes [2] provide the canonical closed-form valuation framework for European options under constant volatility and other standard assumptions. When those assumptions are relaxed, numerical approaches become more important. Monte Carlo methods are flexible because they can accommodate general stochastic dynamics and path-dependent features, but the price estimate is subject to simulation error and typically requires repeated path generation [4]. Heston [3] extends the modeling framework by allowing instantaneous variance to follow a mean-reverting stochastic process correlated with the asset-price shock.

2.2. Learning-Based Derivative Pricing

Hutchinson et al. [1] established an early learning-network approach to derivative pricing. Their work is conceptually close to the present study in that model-implied prices are treated as a function to be learned and then evaluated rapidly. Subsequent research has increasingly emphasized stochastic-volatility and high-dimensional settings. Hainaut and Casas [5] formulate Heston pricing with a physics-inspired neural network based on the Feynman-Kac equation. Funahashi [6] studies methods that learn pricing residuals relative to asymptotic or quasi-process approximations and argues that residual learning can improve stability and reduce the amount of training data required. Zhang, Amici, and Morandotti [8] develop deep differential networks for fast Heston calibration by learning prices and sensitivities.

2.3. Surrogate Models and the Computational Argument

A common feature of this literature is the separation between offline computation and online evaluation. Neural networks can be expensive to train because reference labels must first be generated, but once fitted they can evaluate the learned operator rapidly. Chen et al. [7] formalize this idea as deep surrogates for financial structural models and demonstrate order-of-magnitude computational gains in a high-dimensional option-pricing application. The current study is narrower: it does not seek a general structural-surrogate framework, but instead evaluates the economic break-even point of a simple feed-forward surrogate under a controlled Heston Monte Carlo experiment.

This positioning is important for interpretation. The paper does not claim that neural networks are a superior theory of option pricing. It evaluates whether they are useful as an engineering layer between a costly numerical pricing method and repeated downstream queries. The contribution is therefore empirical and computational: accuracy must be measured against the reference model, while usefulness depends on reuse.

3. Theoretical Framework

3.1. Black-Scholes Model

For a European call on a non-dividend-paying asset, the Black-Scholes price is

$$C = S_0 N(d_1) - K \exp(-rT) N(d_2), \quad (1)$$

$$d_1 = [\ln(S_0/K) + (r + \sigma^2/2)T] / (\sigma \sqrt{T}), \quad d_2 = d_1 - \sigma \sqrt{T}. \quad (2)$$

Here S_0 is the spot price, K the strike, T time to maturity, r the continuously compounded risk-free rate, σ volatility, and N the standard normal cumulative distribution function. Because the pricing formula is closed form, its computational cost is already low. The neural network is therefore used primarily as a controlled function-approximation benchmark.

3.2. Monte Carlo Valuation

Under the Black-Scholes risk-neutral dynamics, the asset price follows a geometric Brownian motion. For a simulated set of terminal prices S_{T^m} , the European call value is estimated as

$$C_{MC} = \exp(-rT) (1/M) \sum_m \max(S_{T^m} - K, 0). \quad (3)$$

For Heston, the same principle is applied after simulating the joint asset and variance processes. Monte Carlo remains flexible, but simulation error falls only with additional computation; this makes repeated valuation a natural setting in which a reusable surrogate may be valuable.

3.3. Heston Stochastic Volatility

The Heston model specifies

$$dS_t = r S_t dt + \sqrt{V_t} S_t dW_t^S, \quad (4)$$

$$dV_t = \kappa(\theta - V_t)dt + \xi \sqrt{V_t} dW_t^V, \quad \text{corr}(dW_t^S, dW_t^V) = \rho. \quad (5)$$

The parameter vector supplied to the neural network is $x_H = (S_0, V_0, K, T, r, \kappa, \theta, \xi, \rho)$. V_t denotes instantaneous variance; κ is the mean-reversion speed, θ the long-run variance, ξ the volatility-of-variance parameter, and ρ the correlation between the two Brownian shocks.

3.4. Neural-Network Surrogate

The network approximates the pricing operator with $f_\theta(x_H)$ approximately equal to $C_H(x_H)$. Once the parameters θ are estimated offline, each new pricing query requires only a forward pass. The economic value of this substitution depends on the one-time training cost, the per-query inference cost, the per-query numerical-pricing cost, and the number of future queries.

4. Data and Methodology

4.1. Synthetic Design

Synthetic data are used to isolate model behavior from market microstructure, quote noise, bid-ask effects, and calibration error. The Black-Scholes sample contains 20,000 independently generated parameter vectors covering spot price, strike, maturity, interest rate, and volatility. Black-Scholes prices provide exact synthetic labels under the model. The Heston experiment contains 8,000 independently generated nine-dimensional parameter vectors. The reported Heston training labels use 5,000 antithetic Monte Carlo paths and 100 time steps per option. A fixed random seed controls the principal dataset construction.

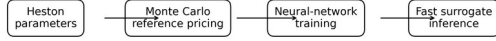


Figure 1. Surrogate-pricing workflow used in the study. Model parameters are mapped to numerical reference prices offline; the trained neural network is then used for repeated fast valuation.

4.2. Train, Validation, and Test Design

The Heston sample is split into training and held-out test data, with a further validation split drawn from the training portion. Standardization parameters are estimated using the training data only and then applied to validation and test observations. Three independent neural-network initializations are estimated to quantify model-seed variability. The selected model is the seed with the lowest test-set MAE, while mean and 95% confidence intervals across all three seeds are reported as a robustness summary.

4.3. Neural-Network Architecture and Training

The final Heston surrogate is a fully connected multilayer perceptron with architecture 9-128-128-64-32-1. ReLU activations are used in hidden layers and a scalar output produces the option price. The optimizer is Adam with learning rate 0.001. Training uses Huber loss rather than squared error, with a maximum of 1,000 epochs and early stopping after 50 validation epochs without sufficient improvement. The architecture is intentionally moderate; the purpose is to improve approximation quality without turning the study into an extensive architecture-search exercise.

4.4. Monte Carlo Reference Engine

The Heston Monte Carlo engine is compiled with Numba and uses antithetic shocks. Variance is kept non-negative through a full-truncation style correction in the simulation update. The higher-accuracy 5,000-path, 100-step simulation is used to generate training labels. The repeated-pricing benchmark is held at 3,000 paths and 100 steps for both methods, so the accuracy upgrade does not silently change the computational benchmark.

4.5. Evaluation Metrics

The primary accuracy metrics are mean absolute error (MAE) and root mean squared error (RMSE):

$$\text{MAE} = (1/n) \sum_i |C_i^{\text{NN}} - C_i^{\text{ref}}|, \quad (6)$$

$$\text{RMSE} = \sqrt{(1/n) \sum_i (C_i^{\text{NN}} - C_i^{\text{ref}})^2}. \quad (7)$$

Normalized MAE is defined as MAE divided by the mean absolute reference price. Raw percentage errors can become unstable when option prices approach zero. Accordingly, the paper reports a relative-error diagnostic only for observations with reference price at least 1 monetary unit; MAE and normalized MAE remain the primary metrics.

4.6. Amortized Computational Cost

Let T_{train} denote one-time network training time, t_{NN} the mean neural-network inference time per option, and t_{MC} the corresponding Monte Carlo cost per option. Cumulative costs for N future pricing queries are

$$T_{\text{NN}}(N) = T_{\text{train}} + N t_{\text{NN}}, \quad T_{\text{MC}}(N) = N t_{\text{MC}}. \quad (8)$$

$$N^* = T_{\text{train}} / (t_{\text{MC}} - t_{\text{NN}}), \quad \text{when } t_{\text{MC}} > t_{\text{NN}}. \quad (9)$$

This break-even calculation is the core economic test of the surrogate. The threshold is an implementation-specific result and changes with hardware, batch size, path count, discretization, and the required pricing accuracy.

5. Empirical Results

5.1. Black-Scholes Benchmark

For the standard at-the-money benchmark $S_0 = K = 100$, $T = 1$, $r = 5\%$, and $\sigma = 20\%$, the analytical Black-Scholes price is 10.4506. The neural network achieves an MAE of 0.5172 and RMSE of 0.6818 across the held-out synthetic sample, corresponding to normalized MAE of 3.03%. The Monte Carlo benchmark produces a mean price of 10.4675 across five runs, with an absolute error of 0.0169 relative to the analytical value. The result confirms that the feed-forward network can learn the pricing surface, but it does not provide a compelling reason to replace the closed-form formula.

Method	Result
Black-Scholes	10.4506; closed form
Monte Carlo	10.4675; error 0.0169; 0.00615 s
NN surrogate	MAE 0.5172; RMSE 0.6818

5.2. Heston Surrogate Accuracy

The final Heston experiment materially reduces prediction error relative to the earlier benchmark used in the project, while simultaneously using a larger training set and higher-quality Monte Carlo labels. The selected seed achieves MAE 0.296790 and RMSE 0.388257. Across three independent seeds, mean MAE is 0.313190 with a 95% confidence interval of 0.274817-0.351564, while mean RMSE is 0.409328 with a 95% confidence interval of 0.361270-0.457386. Mean normalized MAE across seeds is 1.8570%.

Measure	Value
MAE	0.2968; mean 0.3132 [0.2748-0.3516]
RMSE	0.3883; mean 0.4093 [0.3613-0.4574]
Normalized MAE	1.76%; mean 1.86% [1.63-2.08%]
Training time	4.58 s; mean 4.49 s [4.18-4.79]

The final results are not interpreted as evidence that changing the network architecture alone caused the improvement, because the experiment also uses more training observations, more accurate Monte Carlo labels, longer optimization, and Huber loss. Instead, they establish the performance of the complete final surrogate specification adopted in this study.

5.3. Error Across Moneyness

Pricing error remains moderate across the three moneyness regions. OTM options have MAE 0.2532, near-ATM options 0.3244, and ITM options 0.3193. Normalized MAE is 1.50%, 1.92%, and 1.89%, respectively. The restricted relative-error diagnostic, computed only for reference prices at least 1, is 5.09%, 3.05%, and 1.31%. The pattern suggests that the

surrogate is not confined to a narrow ATM region, although the normalized and relative measures should be interpreted with the usual caution for low-price options.

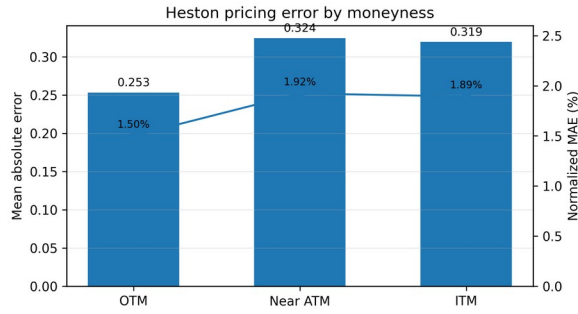


Figure 2. Heston pricing error by moneyness. Bars show MAE; the line shows normalized MAE. Relative errors are not used as the primary metric because low-price options can mechanically generate large percentages.

5.4. Error Across Maturity

MAE ranges from 0.2311 for 0.5-1 year options to 0.3583 for options with maturity above two years. Normalized MAE remains between 1.37% and 2.12% across the four maturity buckets. The results therefore do not indicate a severe concentration of error in short-dated contracts, although the longest-maturity region is the least accurate of the reported buckets.

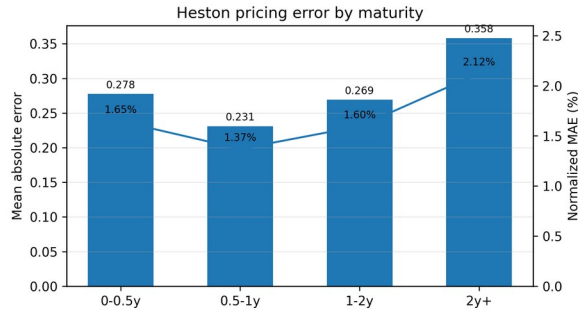


Figure 3. Heston pricing error by maturity. The surrogate maintains similar normalized error across maturity buckets, with somewhat larger absolute error for the longest maturities.

5.5. High-Accuracy Reference Check

To distinguish surrogate approximation error from noise in the 5,000-path training labels, 100 held-out observations are independently revalued using 20,000 antithetic paths and 200 time steps. Against this higher-accuracy reference, the selected network achieves MAE 0.274487 and RMSE 0.356062. This check provides additional evidence that the test-set errors are not driven solely by the finite simulation accuracy used to construct training labels.

5.6. Computational Performance

The repeated-pricing benchmark uses the same held-out parameter vectors for both methods, compiled antithetic Heston Monte Carlo with 3,000 paths and 100 time steps, and warm-up before measurement. Across five Monte Carlo timing runs, the median runtime is 0.906601 seconds; neural-network inference over the same held-out batch has a median runtime of 0.000717 seconds across 20 runs. The median speed ratio is 1,265.1 times. A bootstrap confidence interval for the ratio is 1,162.2-1,357.5 times.

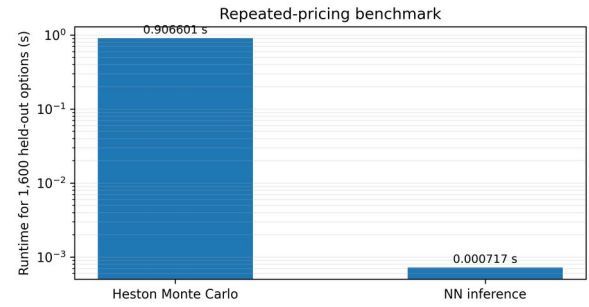


Figure 4. Repeated-pricing benchmark on the held-out Heston batch. The benchmark is implementation-specific and uses compiled antithetic Monte Carlo with 3,000 paths and 100 time steps.

Timing metric	Value
Median batch runtime	MC 0.906601 s; NN 0.000717 s
Mean 95% CI	MC 0.8895-0.9455 s; NN 0.000697-0.000808 s
Per-option runtime	MC 0.00056663 s; NN 0.000000448 s
Speed ratio	1,265.1x; bootstrap 95% CI 1,162.2-1,357.5x

5.7. Amortized Break-Even

The selected model requires 4.577 seconds of one-time training. Using the measured per-option costs, the estimated break-even point is 8,084 pricing queries. Below this level, the numerical method can remain cheaper once training is included; above it, the cumulative inference advantage dominates under the benchmark assumptions.

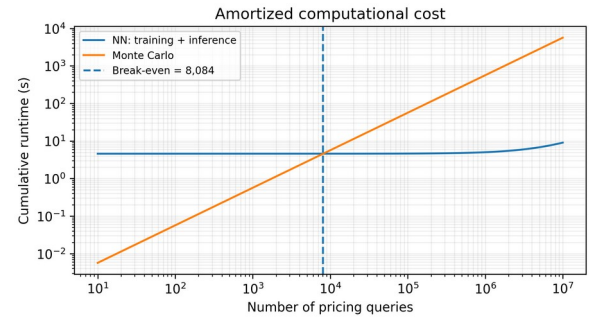


Figure 5. Amortized computation cost. The break-even point is approximately 8,084 pricing queries using the measured training time and benchmark per-option runtimes.

6. Discussion

6.1. Black-Scholes as a Negative Benchmark

The Black-Scholes experiment is deliberately a negative result. The network can approximate the nonlinear pricing surface, but the analytical formula is already deterministic, exact under its assumptions, and inexpensive. Introducing a trained approximation therefore adds a new source of error and an offline training cost without addressing a meaningful computational bottleneck. This finding is consistent with the original motivation in the learning-network literature: surrogate methods are most attractive when the underlying pricing problem is not already cheap and closed form [1].

6.2. Why Heston Is the Stronger Use Case

The Heston experiment changes the economic comparison because the reference price is produced numerically. A downstream calibration, scenario-analysis, or portfolio-valuation process may need to evaluate related parameter vectors many times. In that setting, the network transforms an expensive repeated numerical calculation into a fast forward pass. The measured break-even analysis makes the trade-off explicit rather than treating inference speed as sufficient evidence of usefulness.

6.3. Accuracy-Speed Trade-off

The final surrogate combines several modest improvements rather than relying on a highly complex architecture. Higher-quality labels reduce the amount of simulation noise presented to the network; Huber loss reduces sensitivity to large residuals; increased model capacity allows a more flexible approximation; and longer training with early stopping allows the optimization to continue while validation performance improves. The resulting 1.76% normalized MAE for the selected seed and 0.2745 MAE against the high-accuracy revaluation provide evidence that the speed advantage is not obtained simply by accepting very large pricing errors.

6.4. Relationship to Recent Literature

The results should be interpreted alongside recent research that increasingly treats neural networks as computational surrogates rather than standalone pricing theories. Hainaut and Casas [5] demonstrate a Heston-specific physics-informed approach. Funahashi [6] shows that residual-learning constructions can improve stability and reduce training-data requirements. Chen et al. [7] demonstrate that deep surrogates can accelerate high-dimensional structural option-pricing calculations, while Zhang et al. [8] extend the learned-pricing idea toward fast Heston calibration through price sensitivities. Relative to these approaches, the present study intentionally emphasizes a transparent feed-forward architecture and an explicit amortized cost calculation.

7. Robustness and Limitations

7.1. Robustness

The study includes three neural-network seeds, repeated timing runs, a bootstrap confidence interval for the speed ratio, and an independent high-accuracy Monte Carlo check. The consistency of the three seed estimates is notable: MAE ranges from 0.2968 to 0.3275 and RMSE from 0.3883 to 0.4263. Error decomposition by moneyness and maturity further reduces the risk that a favorable aggregate metric is driven by a narrow parameter region.

7.2. Limitations

First, the experiment is entirely synthetic and therefore does not evaluate the surrogate against observed option surfaces, market calibration error, or changing market conventions. Second, Heston labels are Monte Carlo estimates and therefore contain sampling and discretization error, despite the use of higher-accuracy training labels and an independent 20,000-path revaluation. Third, the speed ratio is implementation-specific. Although the benchmark uses compiled and antithetic Monte Carlo rather than a Python-level loop, different hardware, vectorization strategies, GPUs, quasi-Monte Carlo methods, or

path counts would change the measured ratio. Fourth, the study is limited to European calls and does not address American exercise, dividends, jumps, transaction costs, multi-asset payoffs, or production calibration workflows.

Finally, the selected neural network is a direct price approximator. The study does not impose explicit no-arbitrage constraints or model sensitivities, and it does not evaluate extrapolation beyond the training parameter domain. These omissions are deliberate: the purpose is to isolate the computational-surrogate question rather than build a full production pricing engine.

8. Conclusion

This study asks when a neural network becomes a useful option-pricing surrogate rather than merely an alternative prediction model. The evidence supports a conditional answer. For Black-Scholes, the closed-form formula remains the appropriate computational benchmark; the network can learn the pricing surface, but there is no compelling reason to introduce training cost and approximation error in place of a cheap analytical solution.

For Heston stochastic volatility, the computational case is materially stronger. The final surrogate, trained on 8,000 synthetic observations with 5,000-path Monte Carlo labels, achieves selected-seed MAE 0.2968 and RMSE 0.3883, with normalized MAE 1.76%. Against a higher-accuracy 20,000-path, 200-step reference on 100 held-out options, MAE is 0.2745 and RMSE is 0.3561. Under the repeated-pricing benchmark, inference is approximately 1,265 times faster than compiled antithetic Monte Carlo, with a bootstrap 95% interval of 1,162-1,358 times. After including 4.58 seconds of training, the estimated break-even point is approximately 8,084 future queries.

The broader methodological lesson is that machine learning should be introduced into quantitative finance in response to a clearly defined computational or statistical bottleneck. In the setting studied here, the relevant advantage is not that a neural network is more sophisticated than Black-Scholes or Heston. It is that a learned surrogate can amortize an expensive numerical calculation over repeated use. The validity of that trade-off depends on approximation error, reference-price quality, implementation-specific computational cost, and the expected number of future pricing queries.

References

- [1] Hutchinson, J. M., Lo, A. W., & Poggio, T. (1994). A nonparametric approach to pricing and hedging derivative securities via learning networks. *The Journal of Finance*, 49(3), 851-889. doi:10.1111/j.1540-6261.1994.tb00081.x.
- [2] Black, F., & Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637-654. doi:10.1086/260062.
- [3] Heston, S. L. (1993). A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The Review of Financial Studies*, 6(2), 327-343. doi:10.1093/rfs/6.2.327.
- [4] Boyle, P., Broadie, M., & Glasserman, P. (1997). Monte Carlo methods for security pricing. *Journal of Economic Dynamics and Control*, 21(8-9), 1267-1321. doi:10.1016/S0165-1889(97)00028-6.

- [5] Hainaut, D., & Casas, A. (2024). Option pricing in the Heston model with physics inspired neural networks. *Annals of Finance*, 20(3), 353-376. doi:10.1007/s10436-024-00452-7.
- [6] Funahashi, H. (2026). Deep learning for derivatives pricing: a comparative study of asymptotic and quasi-process corrections. *Annals of Operations Research*, 357, 113-157. doi:10.1007/s10479-024-06114-1.
- [7] Chen, H., Didisheim, A., & Scheidegger, S. (2026). Deep surrogates for finance: With an application to option pricing. *Journal of Financial Economics*, 177, 104222. doi:10.1016/j.jfineco.2025.104222.
- [8] Zhang, C., Amici, G., & Morandotti, M. (2024). Calibrating the Heston model with deep differential networks. arXiv:2407.15536.
- [9] Moreno T., J. F., Zapata, C., & Aragon, D. (2024). Neural networks and the nonlinear Feynman-Kac theorem applied to financial options pricing. *SN Computer Science*, 5, 697. doi:10.1007/s42979-024-02931-0.